SOFTWARE ARCHITECTURE DESIGN CONCEPTS AND STYLES

COMP6100001 - SOFTWARE ENGINEERING
SESSION 09

UNIVERSITAS BINA NUSANTARA

SUBJECT MATTER EXPERT
Kenny Jingga, S.Kom., M.T.

SESSION 09 – SOFTWARE ARCHITECTURE DESIGN CONCEPTS AND STYLES

LEARNING OBJECTIVES

At the end of this session, students will be expected to be able to:

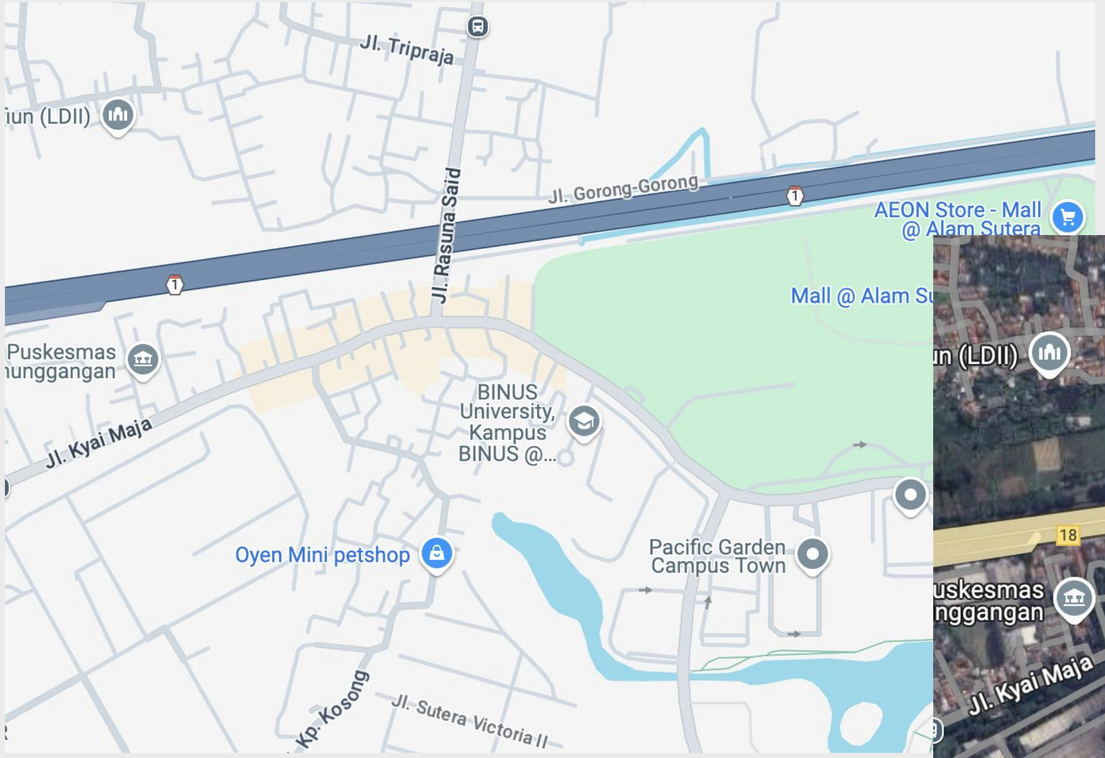
- Understand the Purpose and Importance of Software Architecture
- Describe Key Architectural Styles and Pattern
- Evaluate Architectural Styles for Various Project Requirements
- Explore Architectural Decision-Making Through Case Studies

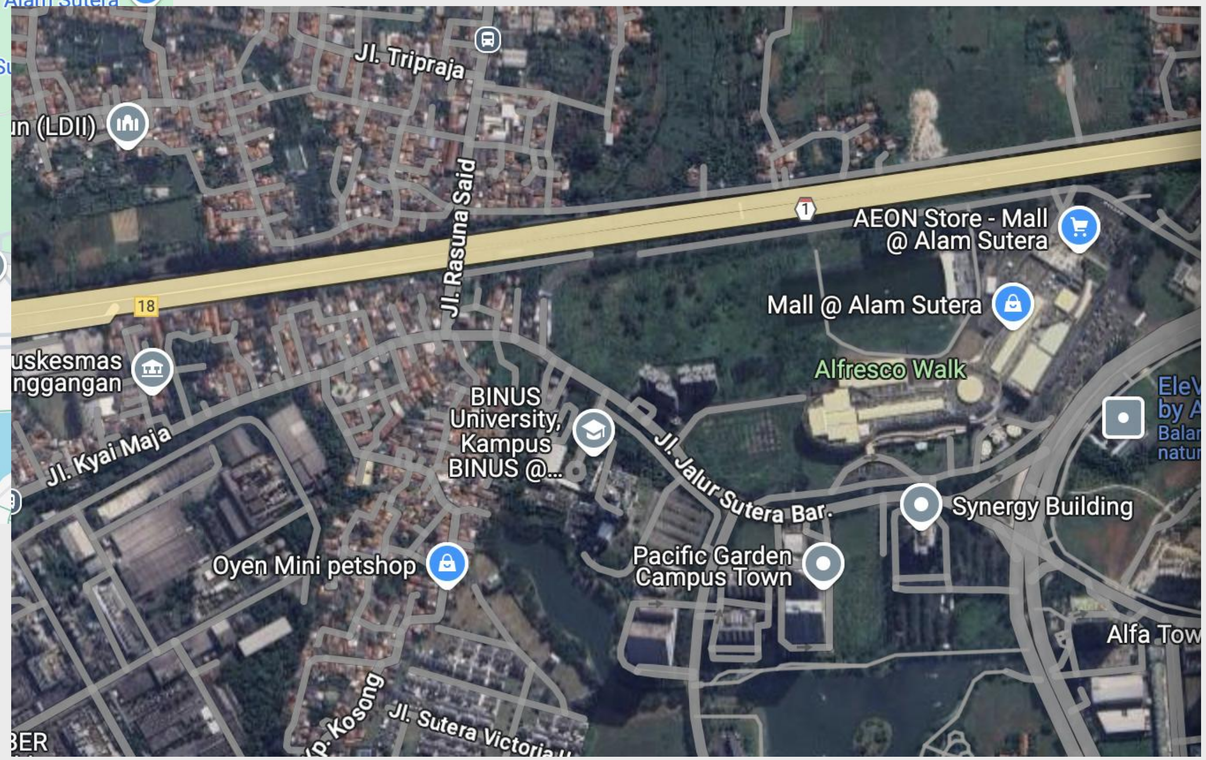
SESSION 09 – SOFTWARE ARCHITECTURE DESIGN CONCEPTS AND STYLES

CONTENT OUTLINE

- Architecture concepts
- Architectural styles & patterns
- Evaluation of styles
- Case studies
- Architectural decision-making

VIEWS AND ABSTRACTION





VIEWS AND ABSTRACTION

ABSTRACTED VIEWS FOCUS ON CONVEYING SPECIFIC INFORMATION

- They have a well-defined purpose
- Show only necessary information
- Abstract away unnecessary details
- Use legends/annotations to remove ambiguity
- Multiple views of the same object tell a larger story

INTRODUCTION TO SOFTWARE ARCHITECTURE

DEFINITION

- **Software Architecture** refers to the fundamental structures of a software system and the discipline of creating such structures and systems

- The software architecture representation is not an operational product

- Rather, it is a representation that enables you to
  (1) analyze the effectiveness of the design in meeting its stated requirements,
  (2) consider architectural alternatives at a stage when making design changes is still relatively easy,
  (3) reduce the risks associated with the construction of the software.

The software architecture of a program or computing system is the **structure or structures** of the system, which **comprise software elements**, the **externally visible properties** of those elements, and the relationships among them

[Bass et al. 2003]

DEFINITION

- Abstraction
- Elements: roles, responsibilities, behaviors, properties
- Relationships between elements
- Relationships to non-software elements
    - Hardware, external systems
- Described from many different perspectives (views)

“Architecture is about the important stuff. Whatever that is.” – Ralph Johnson

INTRODUCTION TO SOFTWARE ARCHITECTURE

DESIGN VS ARCHITECTURE: LEVEL OF ABSTRACTION

- Requirements
  - High-level “what” needs to be done
- Architecture (High-level design)
  - High-level “how”, mid-level “what”
- OO-Design (Low-level design, e.g. design patterns)
  - Mid-level “how”, low-level “what”
- Code
  - Low-level “how”

DESIGN VS ARCHITECTURE: QUESTIONS

Design Questions

- How do I add a menu item in Eclipse?
- How can I make it easy to add menu items in Eclipse?
- What lock protects this data?
- How does Google rank pages?
- What encoder should I use for secure communication?
- What is the interface between objects?

Architectural Questions

- How do I extend Eclipse with a plugin?
- What threads exist and how do they coordinate?
- How does Google scale to billions of hits per day?
- Where should I put my firewalls?
- What is the interface between subsystems?

IMPORTANCE

- **Foundation for Development**: serves as a blueprint for both the system and the project, laying out the tasks necessary to be executed by the design and implementation teams.
- **Facilitates Communication**: provides a common language and understanding among stakeholders.
- **Manages Complexity**: breaks down complex systems into manageable components.
- **Supports Quality Attributes**: influences performance, security, maintainability, scalability, and other non-functional requirements.

INTRODUCTION TO SOFTWARE ARCHITECTURE

ROLES OF SOFTWARE ARCHITECTURE IN SOFTWARE ENGINEERING

- **Guides Design and Implementation**: architecture sets constraints on the design to ensure consistency and coherence.
- **Enables Reuse**: architectural patterns and styles allow for reusing proven solutions.
- **Risk Management**: early architectural decisions help identify and mitigate risks.

ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLES

An **architectural style** defines a family of systems in terms of a pattern of structural organization

- **Monolithic Architecture**
- **Layered Architecture**
- **Client-Server Architecture**
- **Model-View-Controller (MVC)**
- **Model-View-ViewModel (MVVM)**
- **Event-Driven Architecture**
- **Service-Oriented Architecture (SOA)**
- **Microservices Architecture**

ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLES : MONOLITHIC ARCHITECTURE

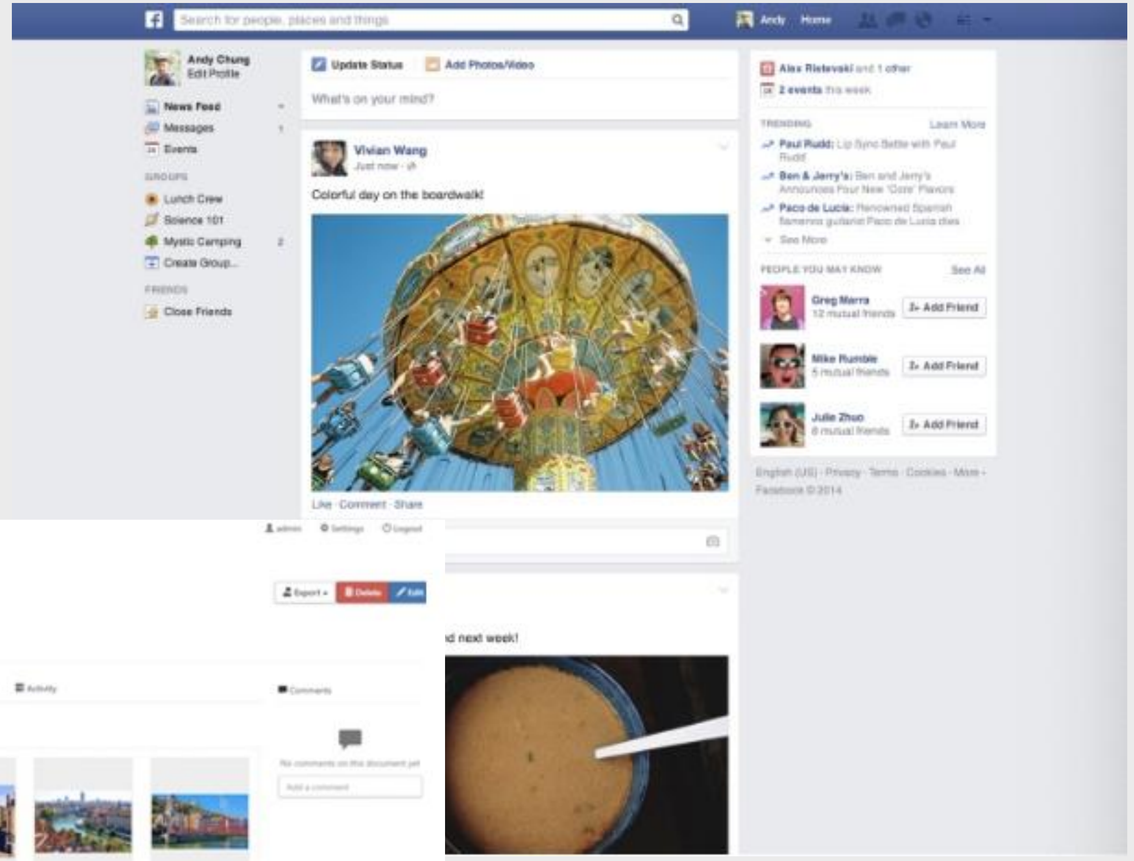
- Monolithic architecture is a concept when complete software is developed as a single unit
- All functionality, features and modules are developed and integrate and deployed in the form of a single entity
- A single platform containing user interface, business logic and database layer on the same page
- Monolithic application has a single shared database for each functionality and every feature combined in a single program as a unified model
- A monolithic architecture is comfortable and easily adopted by small teams and small projects
- A lot of startups and small projects are used to developed using monolithic architecture
- It is good for a project when the modules are interdependent and interconnected

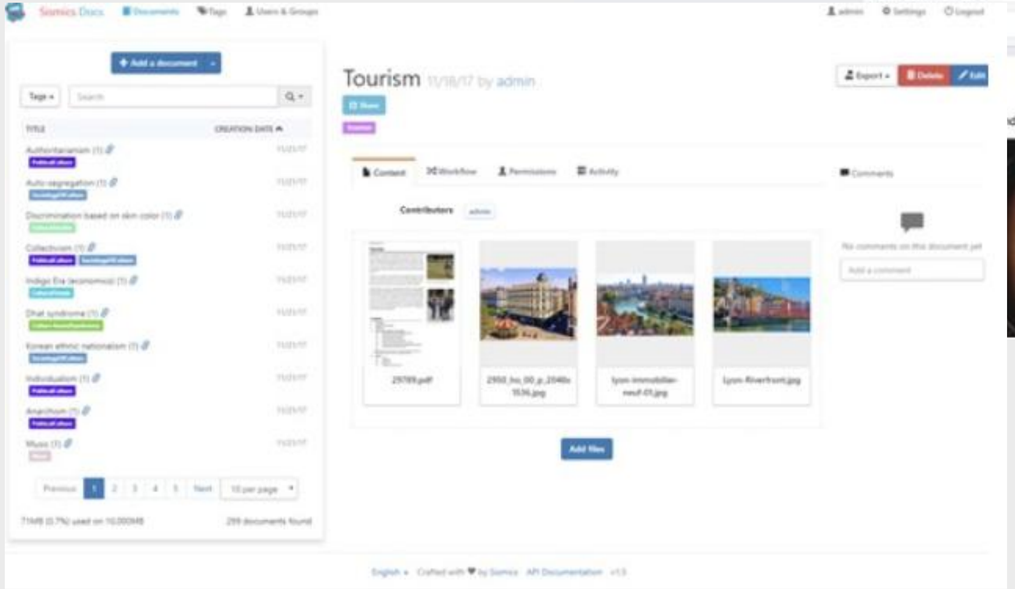
ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLES : MONOLITHS MAKE TRADE-OFFS ON SOFTWARE QUALITY

Several consequences of this architecture on:

- Scalability
- Reliability
- Performance
- Development
- Maintainability
- Evolution
- Testability
- Ownership

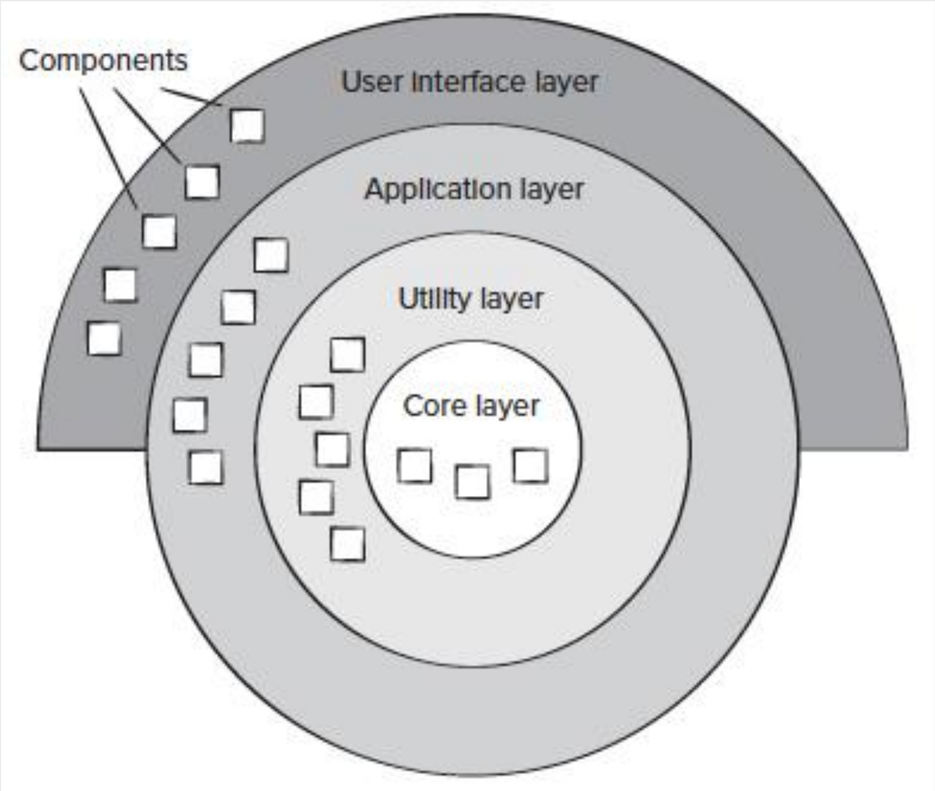




ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: LAYERED ARCHITECTURE

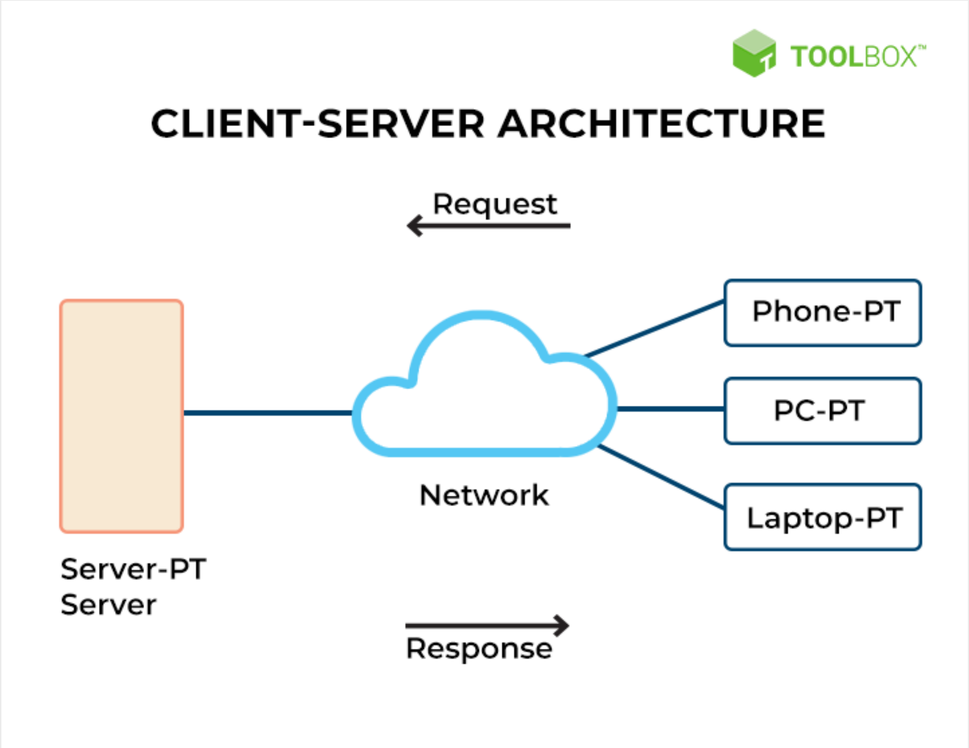
- Organizes the system into layers with related functionality
- Layers:
  - **Presentation Layer**: User interface
  - **Application Layer**: Business logic
  - **Data Access Layer**: Database interactions
  - **Database Layer**: Data storage
- **Advantages**:
  - Separation of concerns.
  - Ease of maintenance and testing.
- **Disadvantages**:
  - Can introduce performance overhead due to layer-to-layer communication.



ARCHITECTURAL STYLES AND PATTERNS

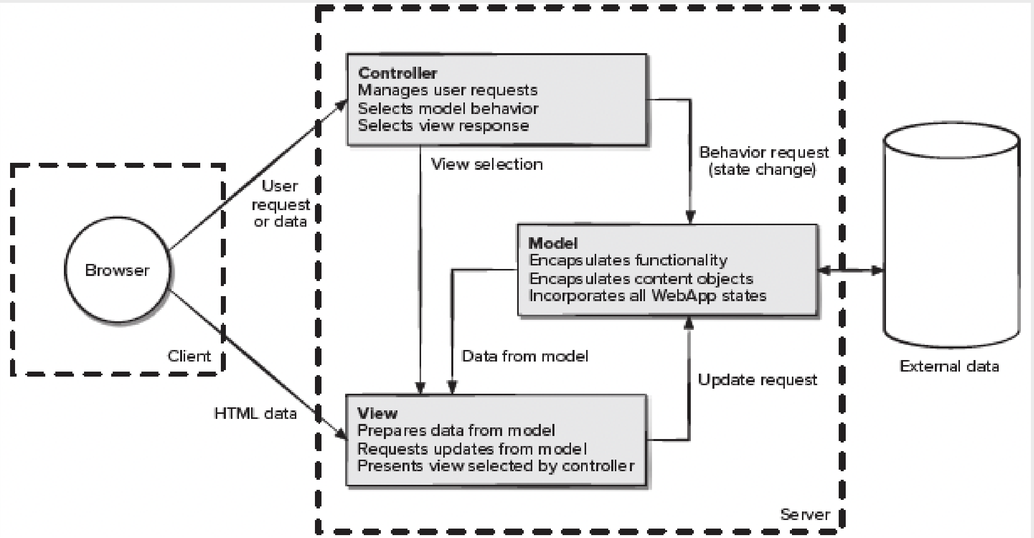
ARCHITECTURAL STYLE: CLIENT-SERVER ARCHITECTURE

- Divides the system into two applications, where the client makes requests to the server.
- Components:
  - **Client**: Front-end application.
  - **Server**: Back-end application that processes requests.
- **Advantages**:
  - Centralized control.
  - Scalability by adding more servers.
- **Disadvantages**:
  - Single point of failure at the server.
  - Network dependency.

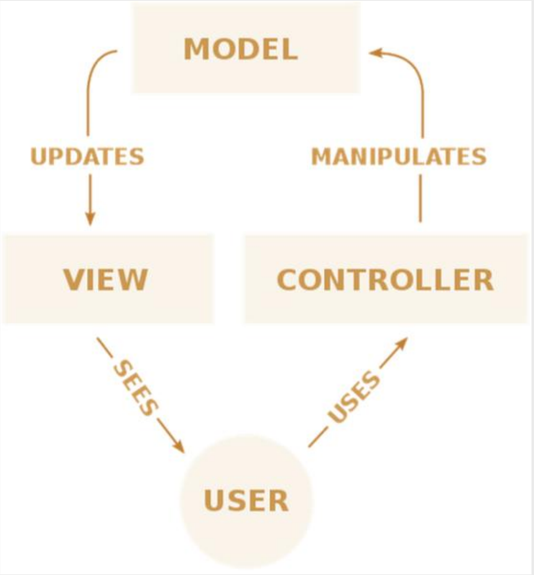


ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: MODEL-VIEW-CONTROLLER (MVC)



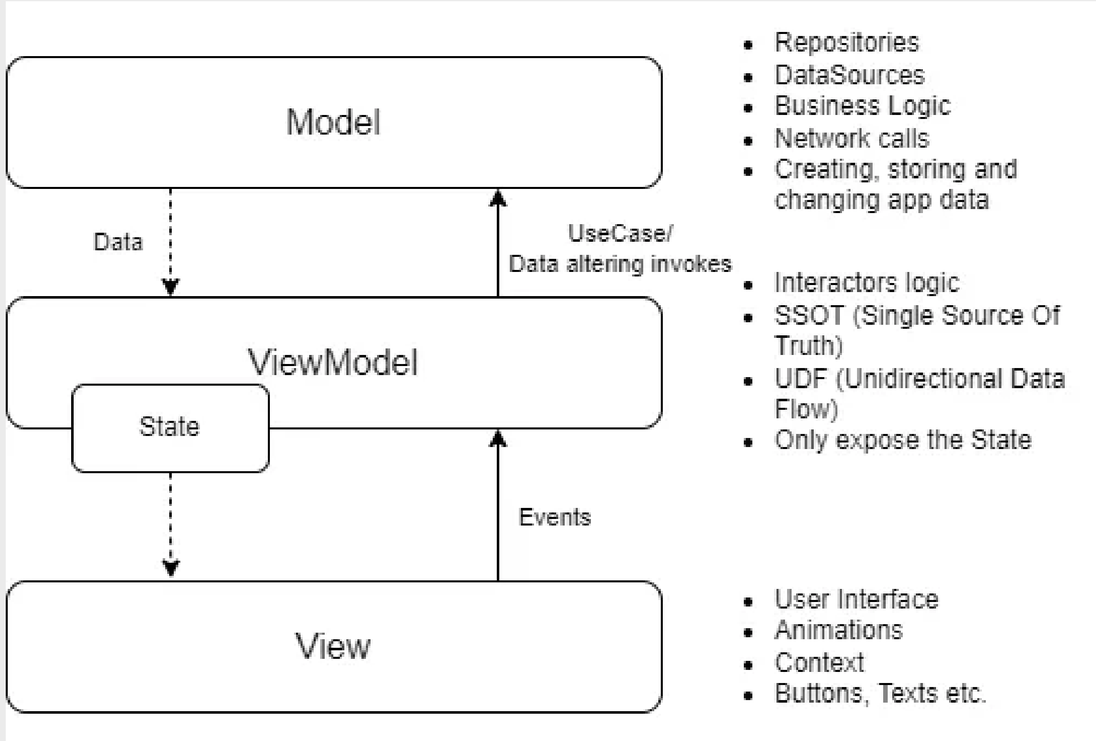
- Separates an application into three main components:
  - **Model**: Manages data and business logic.
  - **View**: Outputs representation of data (UI).
  - **Controller**: Handles input and updates model/view.
- **Advantages**:
  - Facilitates parallel development.
  - Enhances testability and maintainability.
- **Disadvantages**:
  - Can add complexity with multiple components.



ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: MODEL-VIEW-VIEWMODEL (MVVM)

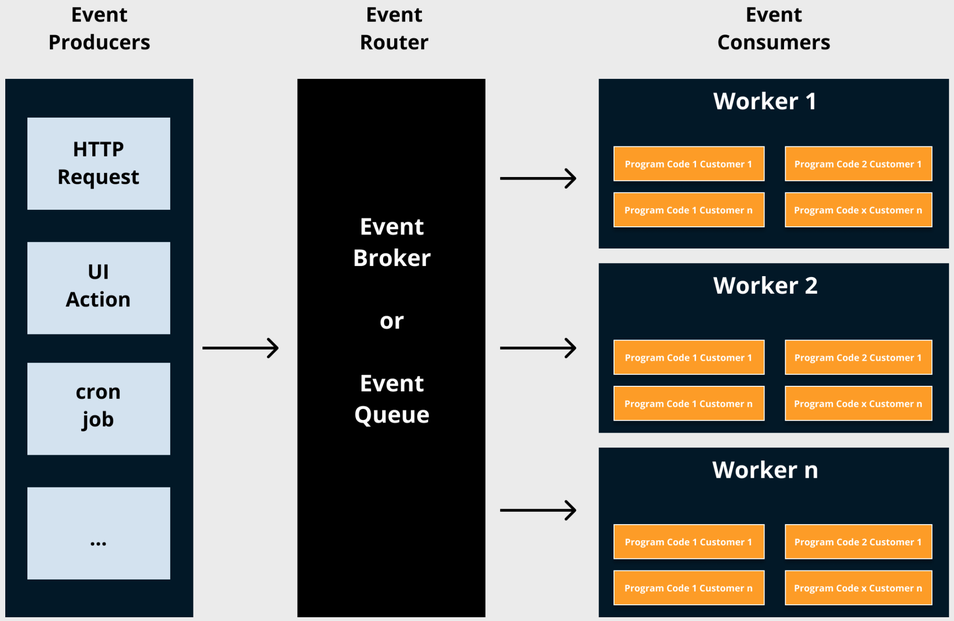
- Enhances MVC by introducing a **ViewModel** that acts as an intermediary between the View and the Model.
- **Components**:
  - **Model**: Data and business logic.
  - **View**: UI elements.
  - **ViewModel**: Handles the logic for the View, facilitates data binding.
- **Advantages**:
  - Promotes a clear separation between the development of UI and the business logic.
  - Facilitates unit testing of the ViewModel.
- **Disadvantages**:
  - May introduce complexity due to data binding and event management.



ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: EVENT-DRIVEN ARCHITECTURE

- Components communicate by producing and responding to events.
- **Components**:
  - **Event Producers:** Generate events.
  - **Event Consumers:** React to events.
- **Advantages**:
  - Highly decoupled.
  - Scalable and flexible.
- **Disadvantages**:
  - Debugging can be difficult.
  - Potential for message loss if not handled properly.

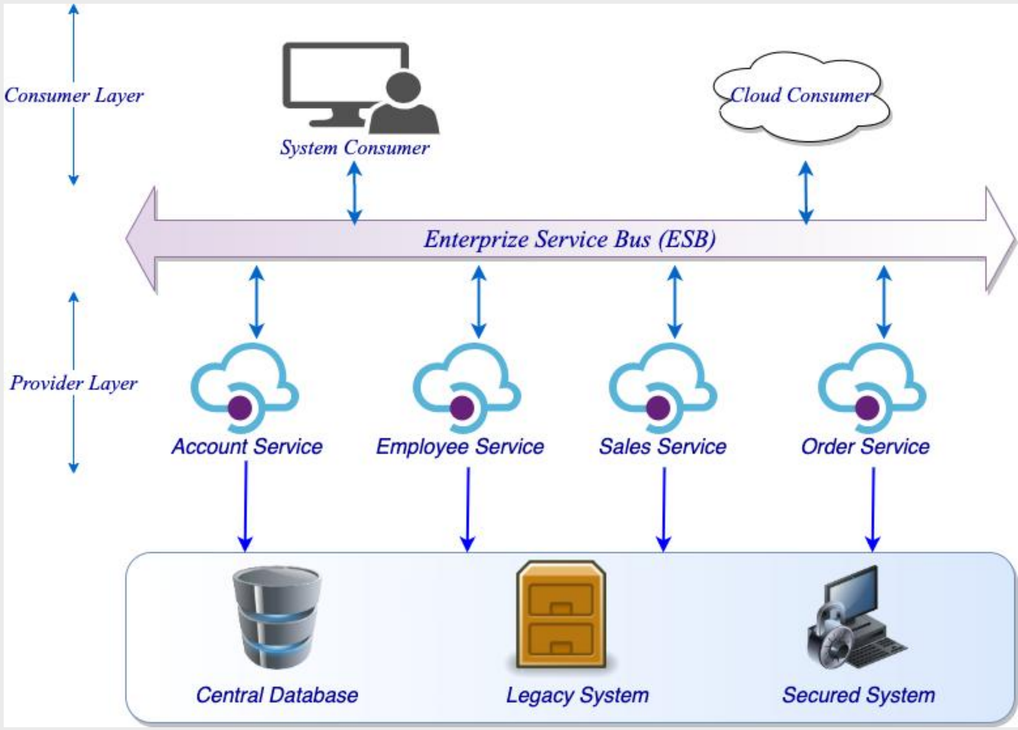


Koyeb

ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: SERVICE-ORIENTED ARCHITECTURE (SOA)

- Services are provided to other components by application components, through a communication protocol over a network.
- **Characteristics**:
  - Reusable services.
  - Standardized service contracts.
- **Advantages**:
  - Promotes interoperability.
  - Facilitates integration with legacy systems.
- **Disadvantages**:
  - Overhead of service communication.
  - Complexity in service management.



ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: SERVICE-ORIENTED ARCHITECTURE (SOA)

- Service-oriented architecture some time is also known as **centralized oriented architecture**, is architecture when multiple, services which are also said as user agents are used or created interaction with a centralized system to make our traditional monolithic less burden and loosely coupled
- A service-oriented architecture is specifically designed to communicate interoperable systems
- Each service provides functionality on abstraction level, consider as a black box, self-contained to avoid the overhead of new development each time
- The communication between them can be done using a central point of connectivity called **Enterprise service bus**
- ESB is taking care of all service and helping them to interact with each other

SERVICE-ORIENTED ARCHITECTURE

WEB BROWSERS: CHROME

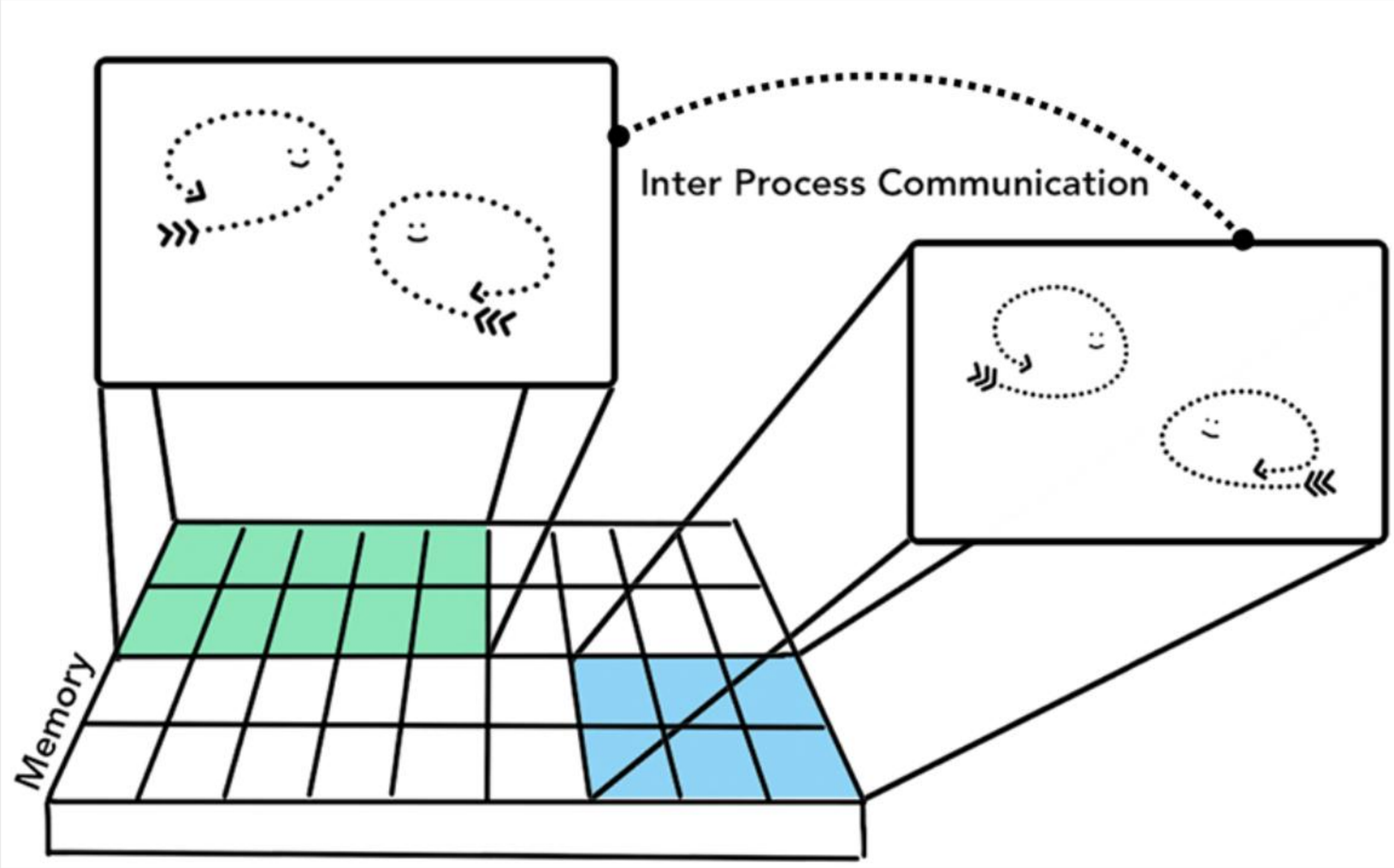
Source: https://developers.google.com/web/updates/2018/09/inside-browser-part1 (CC BY 4.0)

SERVICE-ORIENTED ARCHITECTURE

BROWSER: A MULTI-THREADED PROCESS

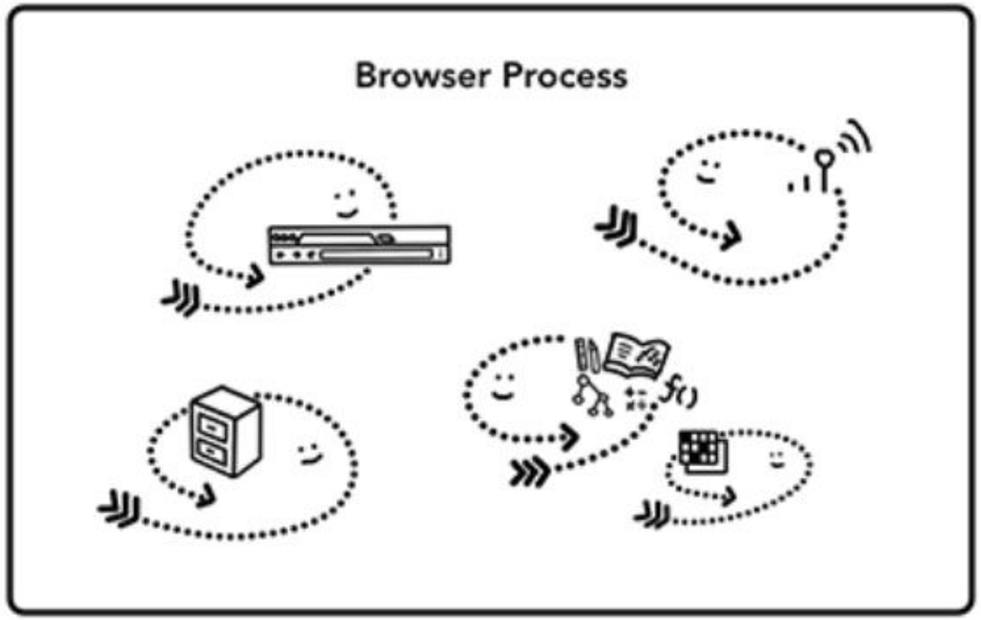
SERVICE-ORIENTED ARCHITECTURE

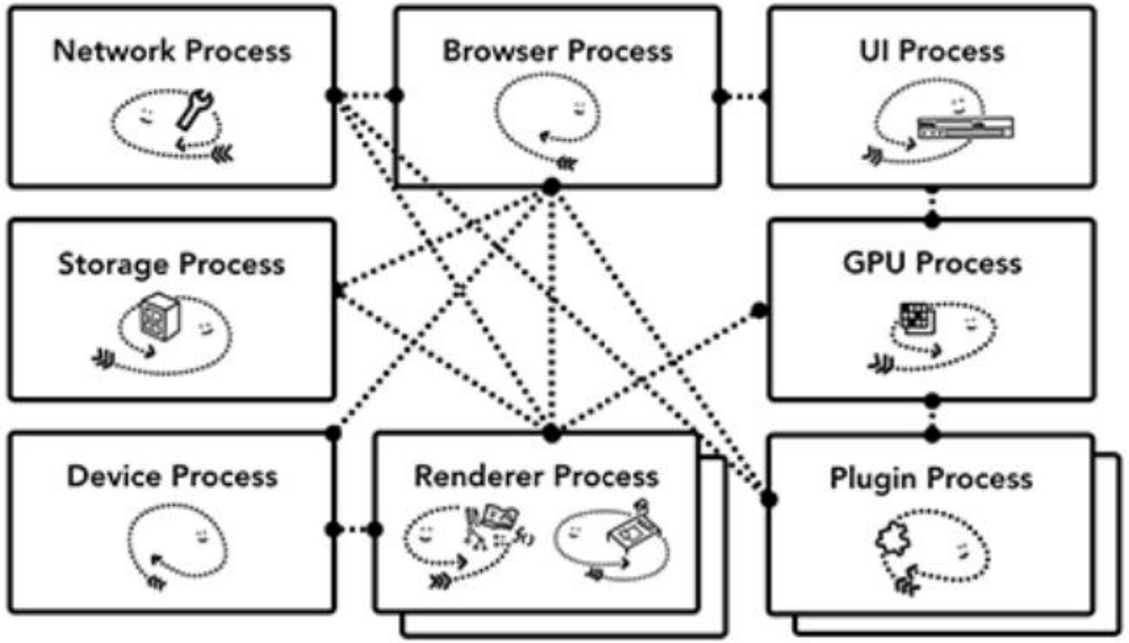
MULTI-PROCESS BROWSER WITH IPC



SERVICE-ORIENTED ARCHITECTURE

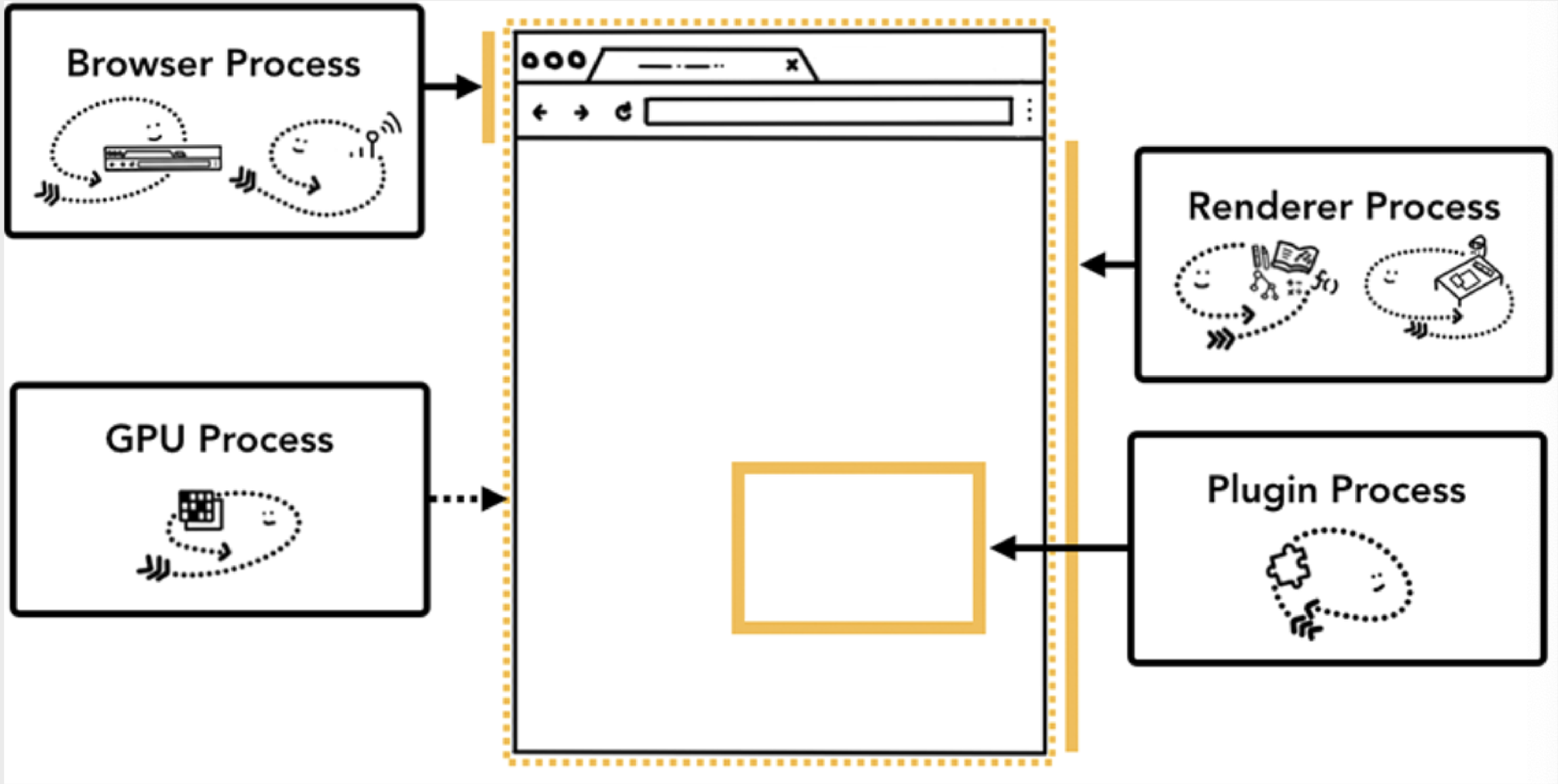
BROWSER ARCHITECTURE





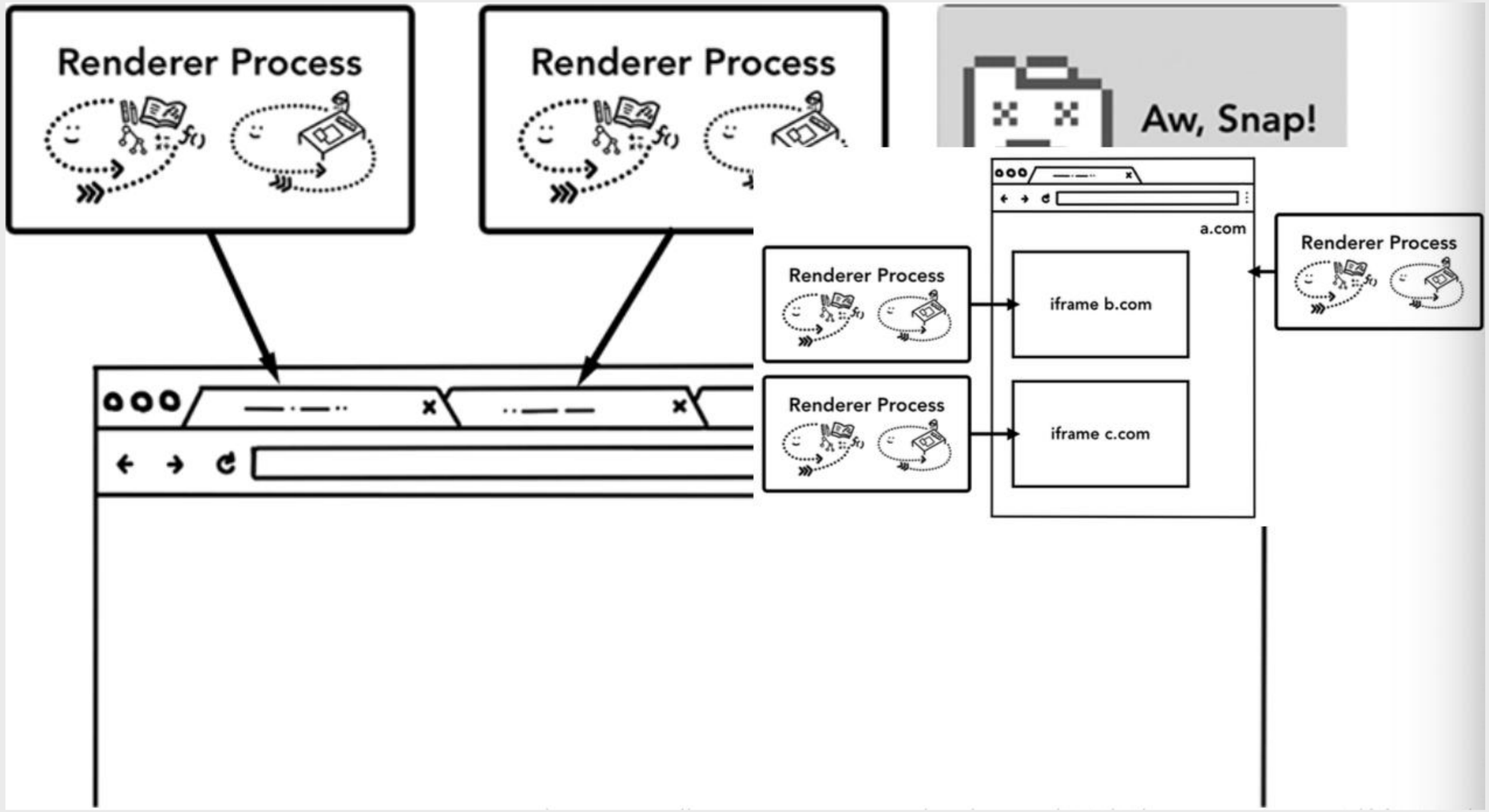
SERVICE-ORIENTED ARCHITECTURE

SERVICE-BASED BROWSER ARCHITECTURE



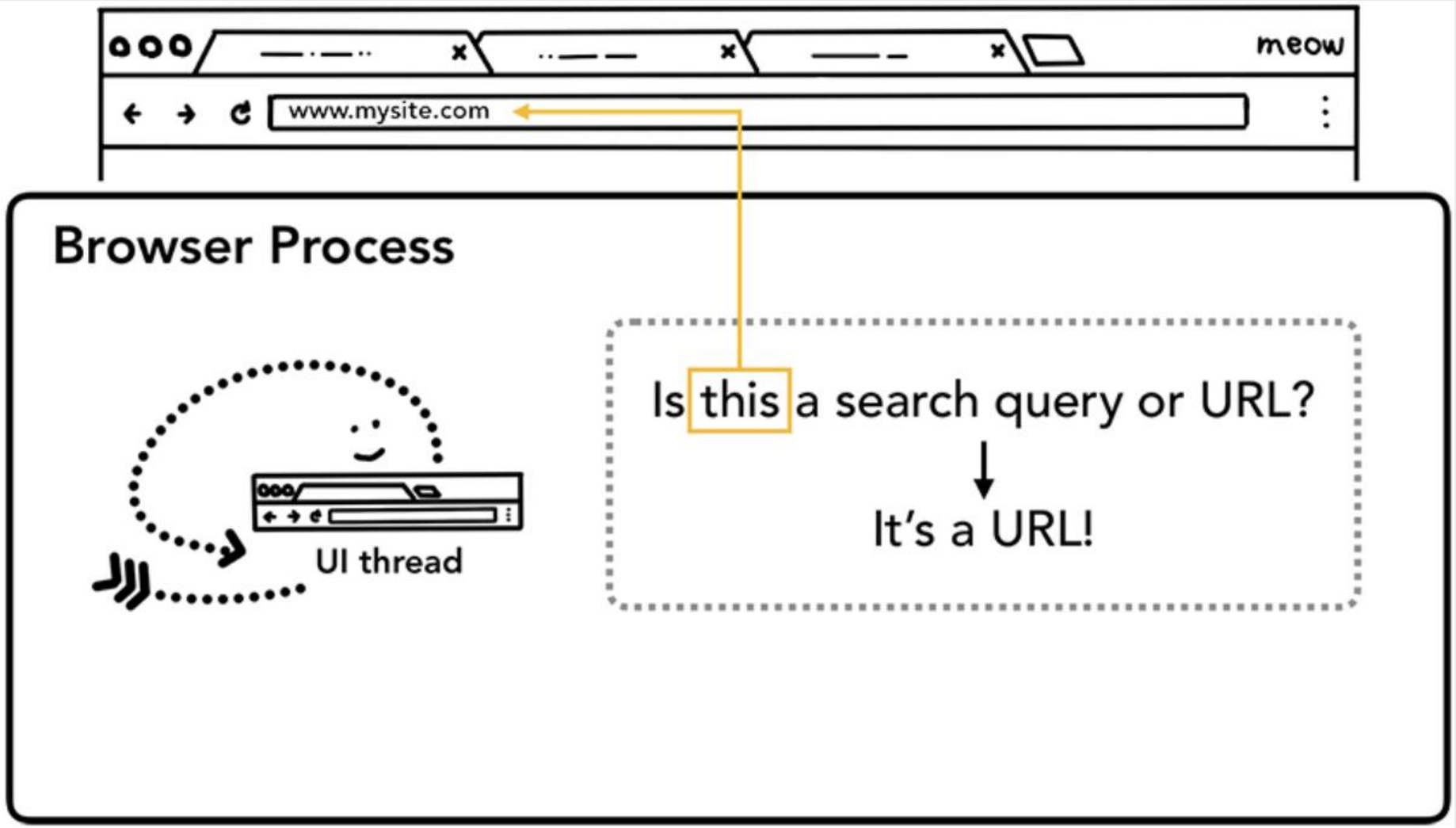
SERVICE-ORIENTED ARCHITECTURE

SERVICE-BASED BROWSER ARCHITECTURE



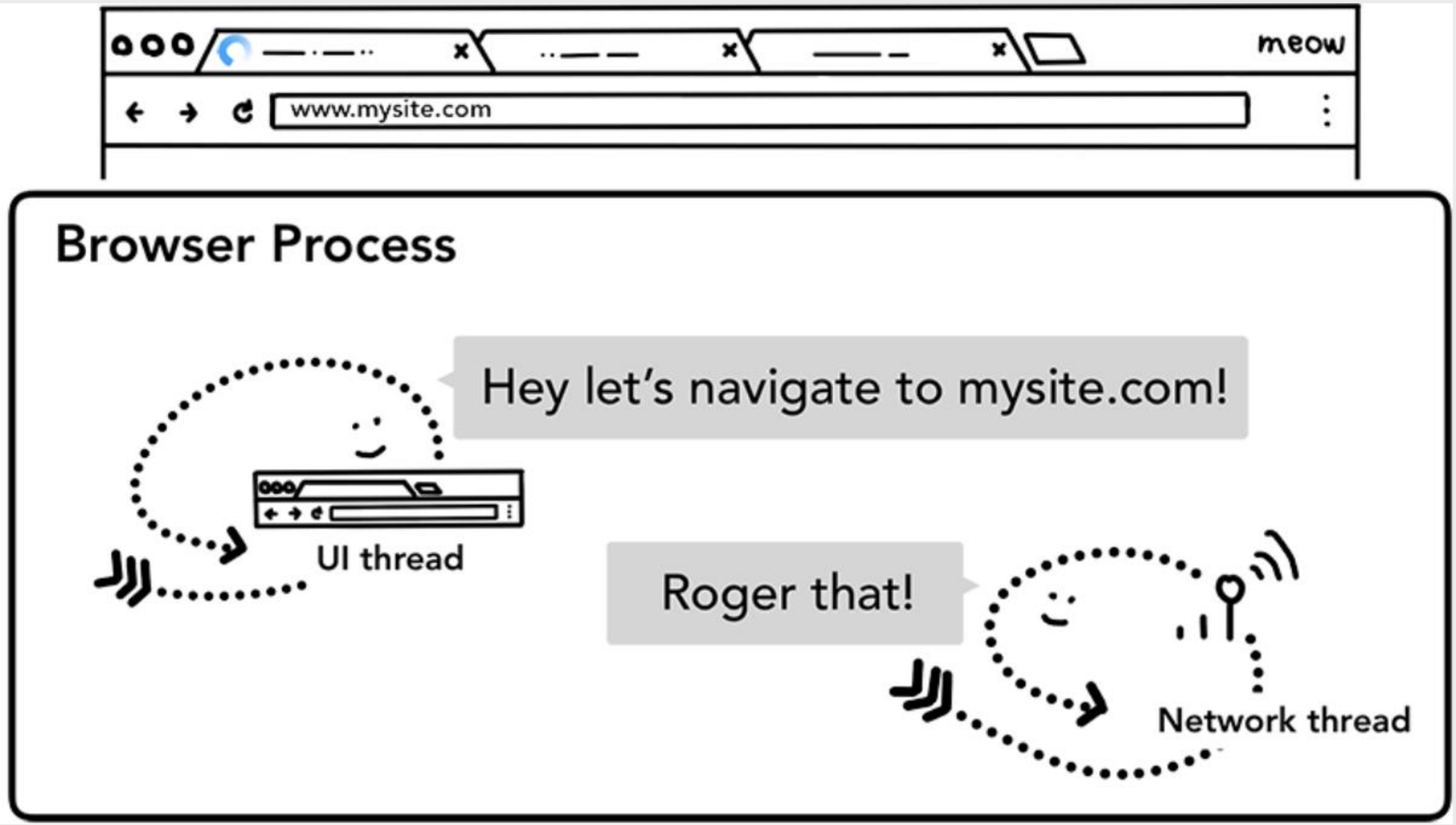
SERVICE-ORIENTED ARCHITECTURE

NAVIGATING TO A WEB SITE USES SERVICE REQUESTS



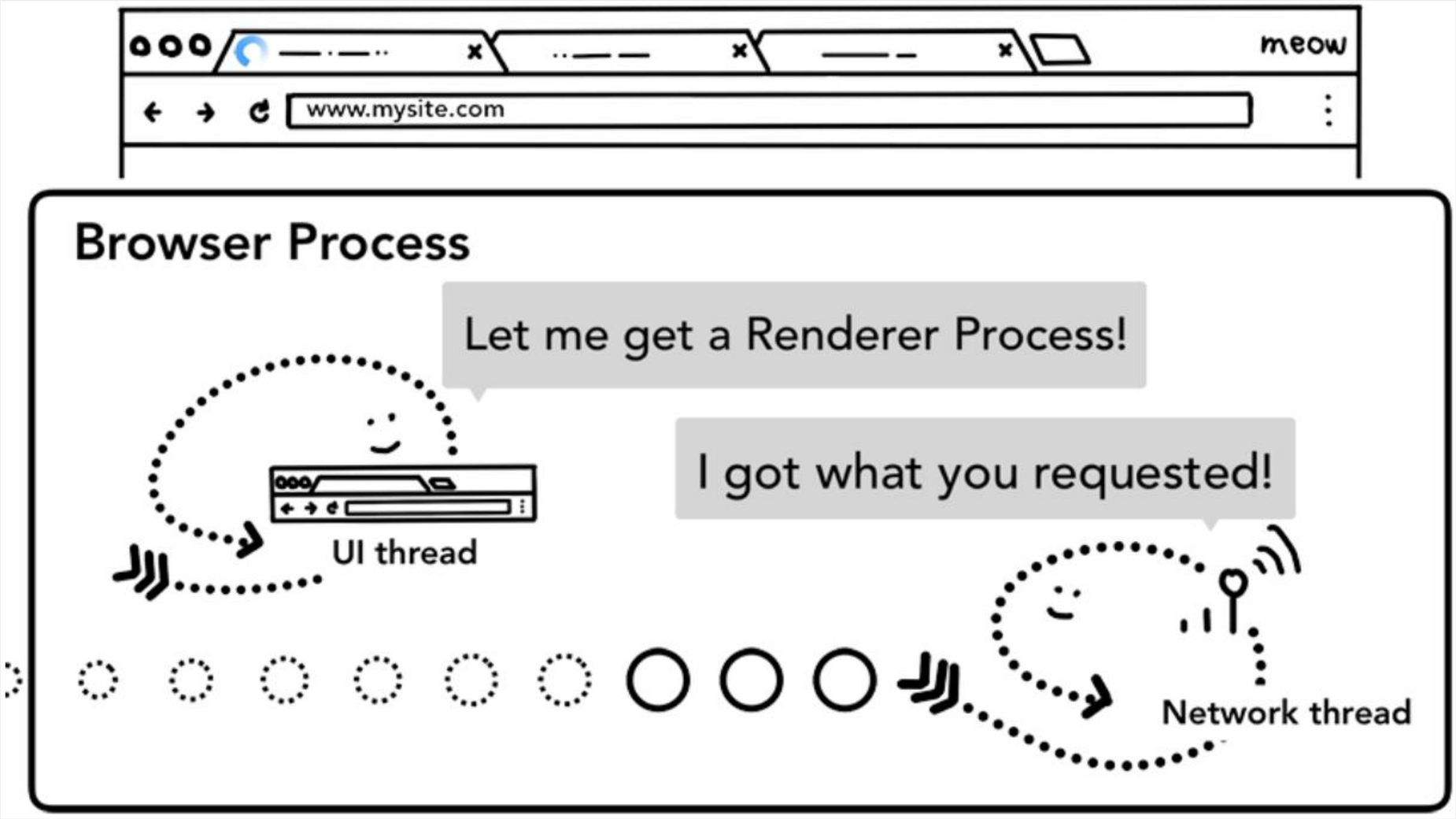
SERVICE-ORIENTED ARCHITECTURE

NAVIGATING TO A WEB SITE USES SERVICE REQUESTS



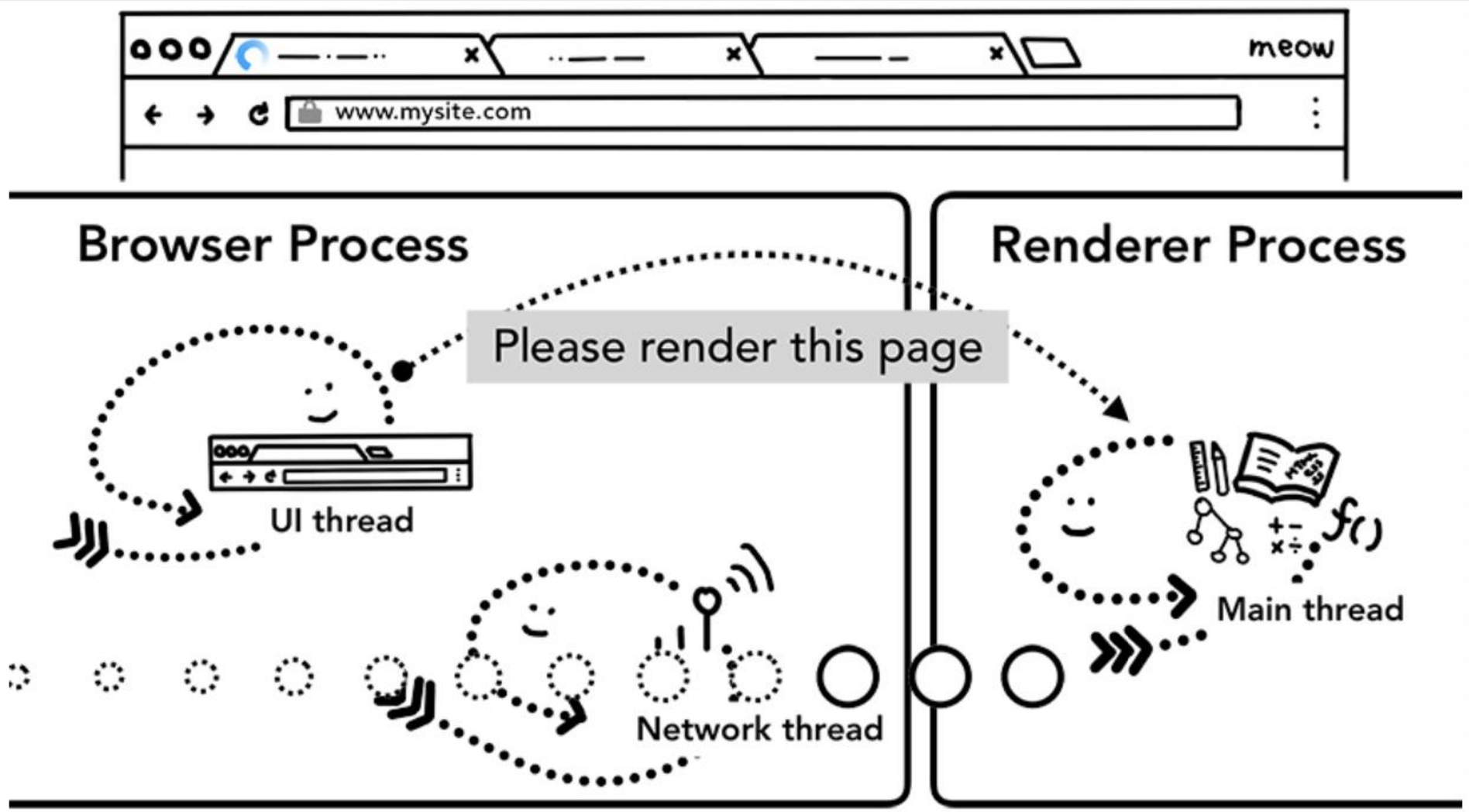
SERVICE-ORIENTED ARCHITECTURE

NAVIGATING TO A WEB SITE USES SERVICE REQUESTS



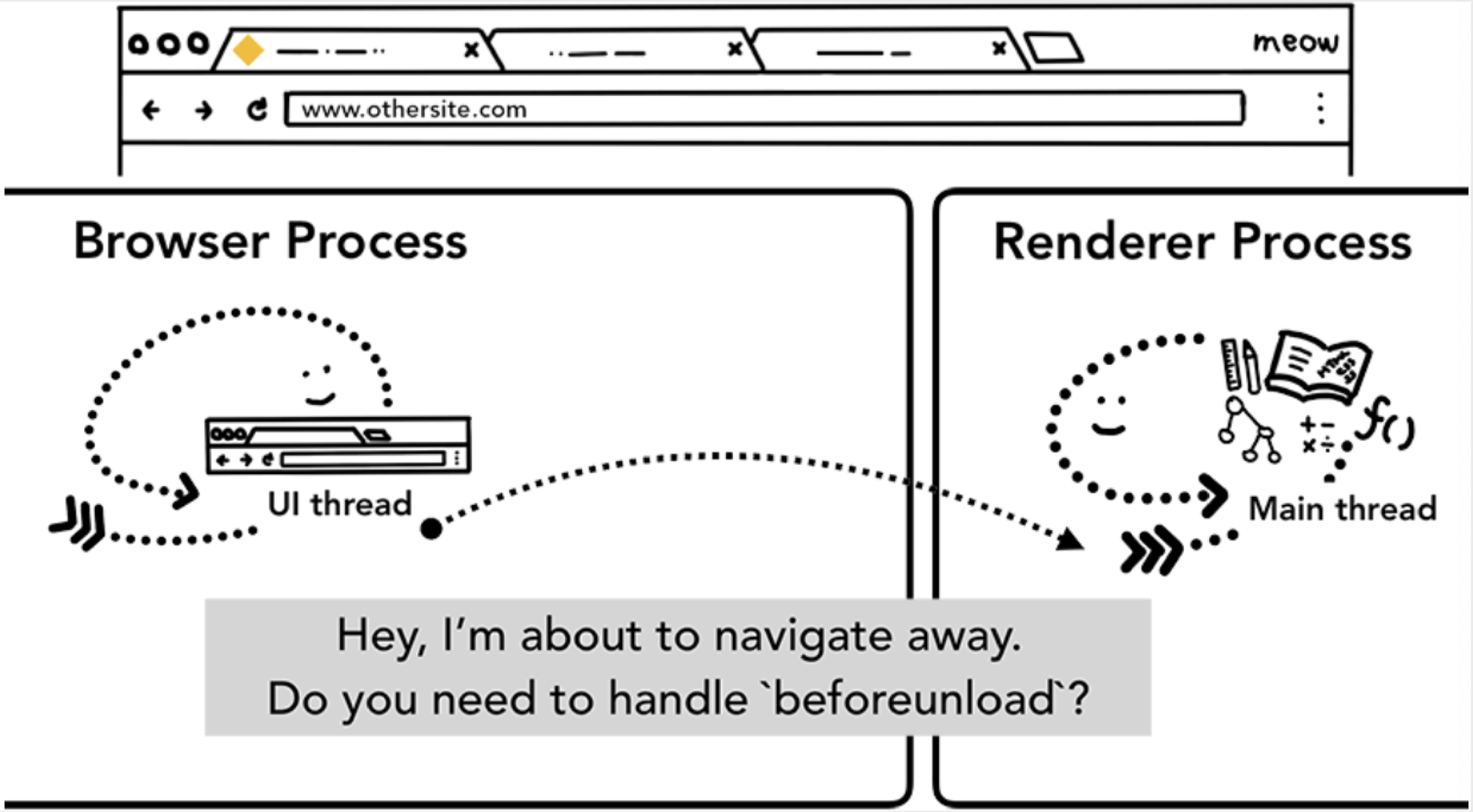
SERVICE-ORIENTED ARCHITECTURE

NAVIGATING TO A WEB SITE USES SERVICE REQUESTS



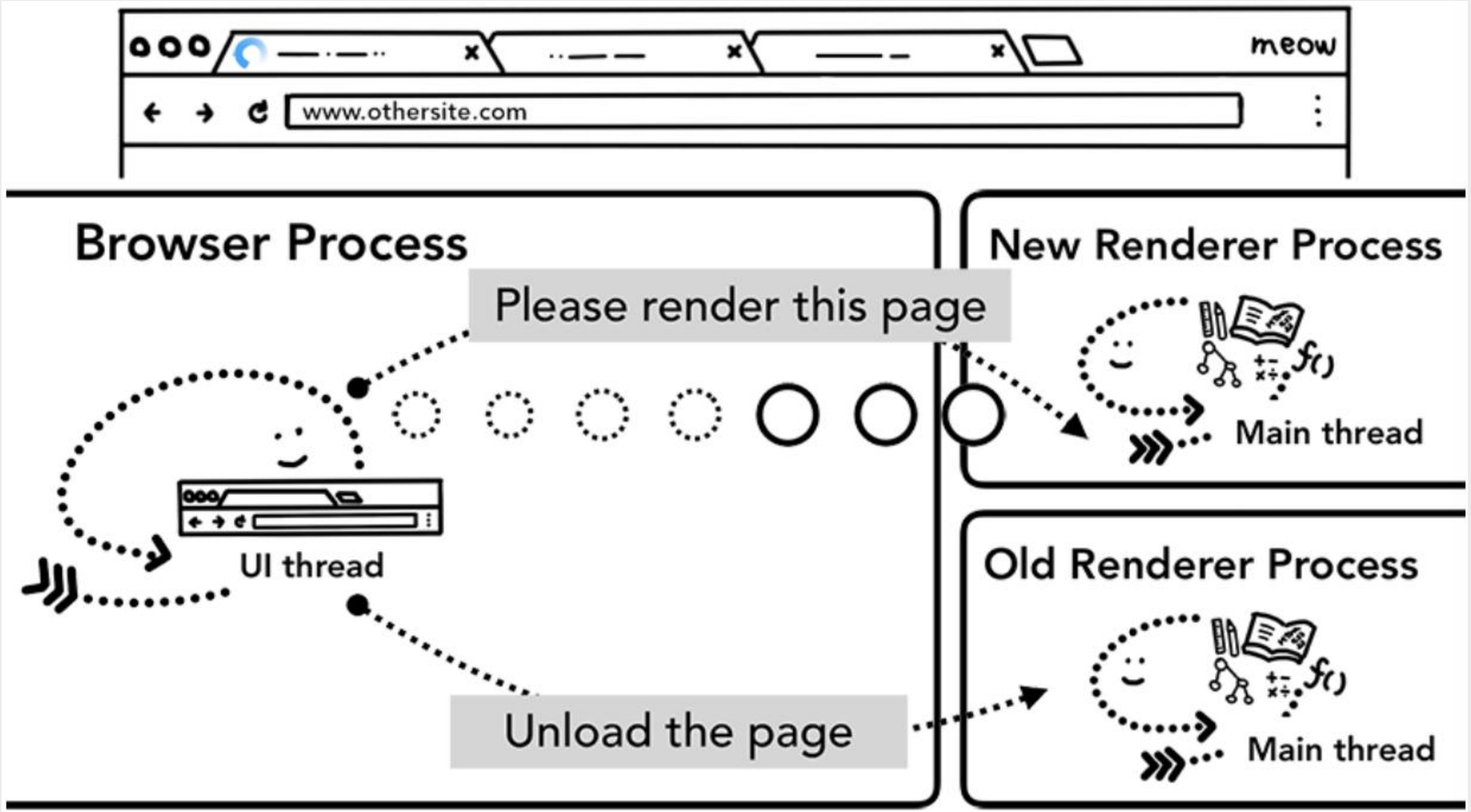
SERVICE-ORIENTED ARCHITECTURE

NAVIGATING TO A WEB SITE USES SERVICE REQUESTS



SERVICE-ORIENTED ARCHITECTURE

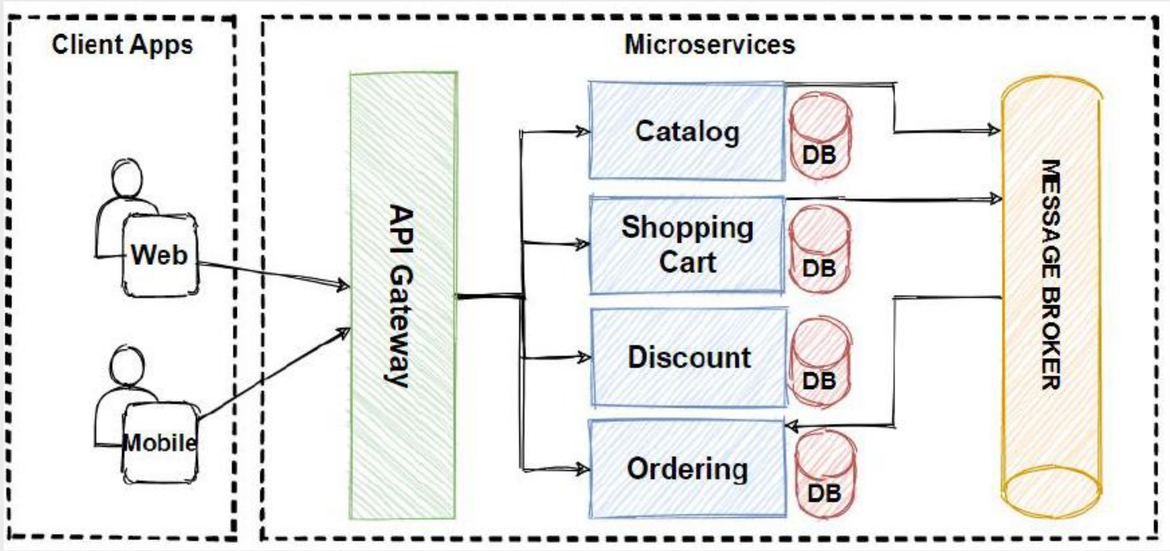
NAVIGATING TO A WEB SITE USES SERVICE REQUESTS



ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL STYLE: MICROSERVICES ARCHITECTURE

- Services are provided to other components by application components, through a communication protocol over a network.
- **Characteristics**:
  - Reusable services.
  - Standardized service contracts.
- **Advantages**:
  - Promotes interoperability.
  - Facilitates integration with legacy systems.
- **Disadvantages**:
  - Overhead of service communication.
  - Complexity in service management.



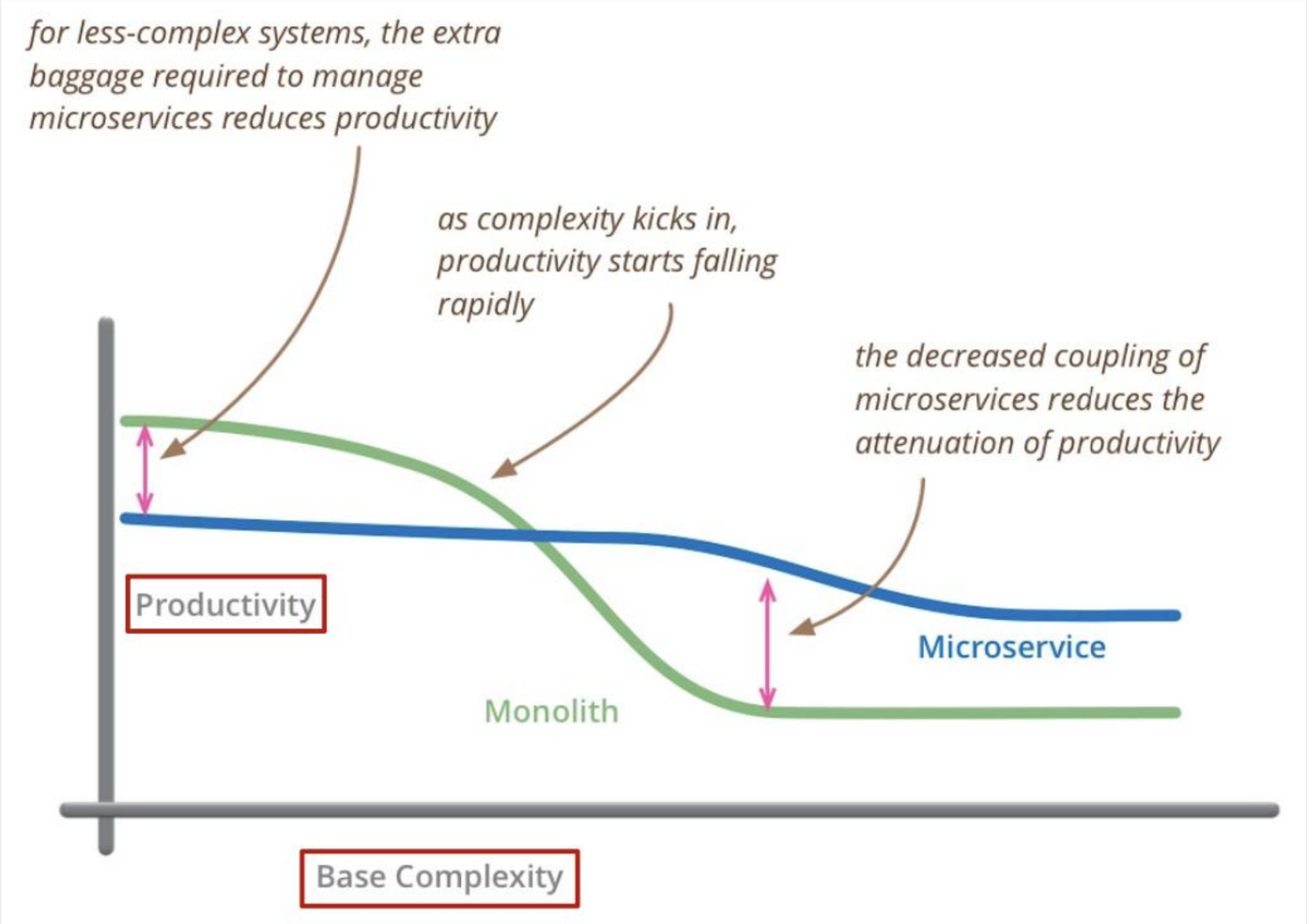
MICROSERVICE ARCHITECTURE

- Microservice architecture also considers as decentralized architecture, is much similar to SOA but not connected with any central service
- Microservice architecture is an architecture which is build using autonomous services which communicate with each other using rest
- Like SOA, microservice not build on such service which can use again any other project as well instead its focus is to make each functionality separate as a service which should be micro and a self-contained, application, business, infrastructure and enterprise layer
- Each service unit has its database
- All those microservice then join into UI microservice to show as a working application
- From UI we call those microservices through rest calls

MICROSERVICE ARCHITECTURE

ARE MICROSERVICES ALWAYS THE RIGHT CHOICE?

- Microservices overhead



MICROSERVICE ARCHITECTURE

CHALLENGES

- Complexities of distributed systems
  - network latency, faults, inconsistencies
  - testing challenges
- Resource overhead, RPCs
  - Requires more thoughtful design (avoid ”chatty” APIs, be more coarse-grained)_
- Shifting complexities to the network
- Operational complexity
- Frequently adopted by breaking down monolithic application
- HTTP/REST/JSON communication
  - Schemas?

ARCHITECTURAL STYLES AND PATTERNS

ARCHITECTURAL PATTERNS VS DESIGN PATTERNS

- **Architectural Patterns**:
  - High-level strategies that concern large-scale components, the global properties, and mechanisms of a system.
  - Examples: Layered architecture, Microservices.
- **Design Patterns**:
  - Solutions to recurring problems in software design at a lower level, focusing on implementation.
  - Examples: Singleton, Observer, Factory Method.

Comparison:

- Scope - **Architectural Patterns**: System-level structure and **Design Patterns**: Component-level design
- Abstraction Level - **Architectural Patterns**: High and **Design Patterns**: Low
- Focus: **Architectural Patterns**: Overall system organization and **Design Patterns**: Implementation details

EVALUATING ARCHITECTURAL STYLES FOR DIFFERENT PROJECTS

CRITERIA FOR SELECTION

- **Functional Requirements**:
  - Does the style support the required functionality?

- **Non-Functional Requirements (Quality Attributes)**:
  - **Performance**: Response time, throughput.
  - **Scalability**: Ability to handle growth.
  - **Security**: Protection against threats.
  - **Maintainability**: Ease of making changes.

EVALUATING ARCHITECTURAL STYLES FOR DIFFERENT PROJECTS

MATCHING ARCHITECTURE TO REQUIREMENTS

- **Use Cases and Scenarios**:
  - Analyze how the architecture handles specific situations.
- **Architecture Tradeoff Analysis Method (ATAM)**:
  - Evaluate architectural decisions based on quality attribute priorities.

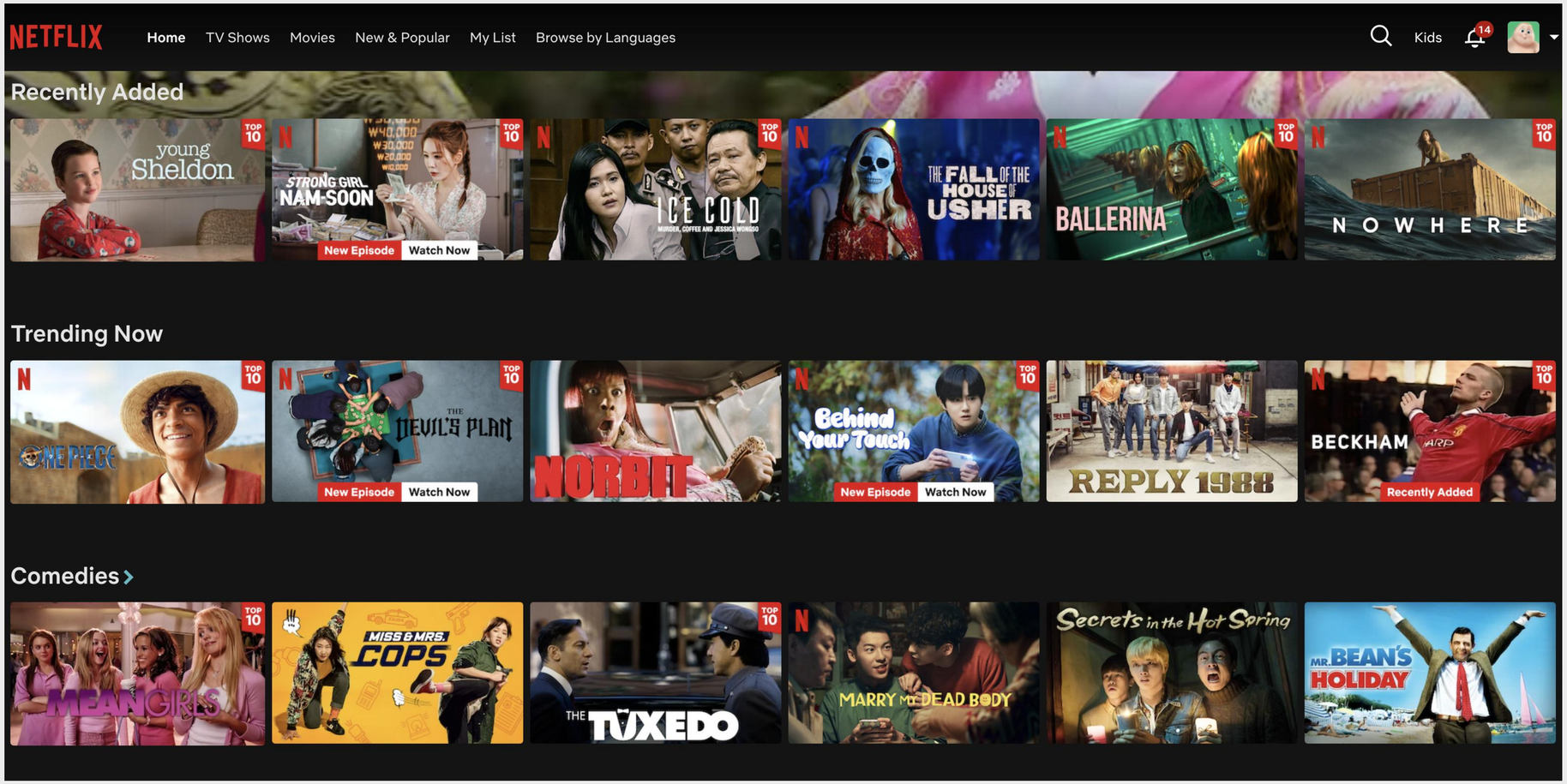
EVALUATING ARCHITECTURAL STYLES FOR DIFFERENT PROJECTS

TRADE-OFFS IN ARCHITECTURAL DECISIONS

- **Complexity vs. Simplicity**:
  - More complex architectures may offer flexibility but can be harder to understand and maintain.
- **Performance vs. Scalability**:
  - Optimization for performance might affect scalability.
- **Time-to-Market vs. Robustness**:
  - Rapid development can lead to technical debt.

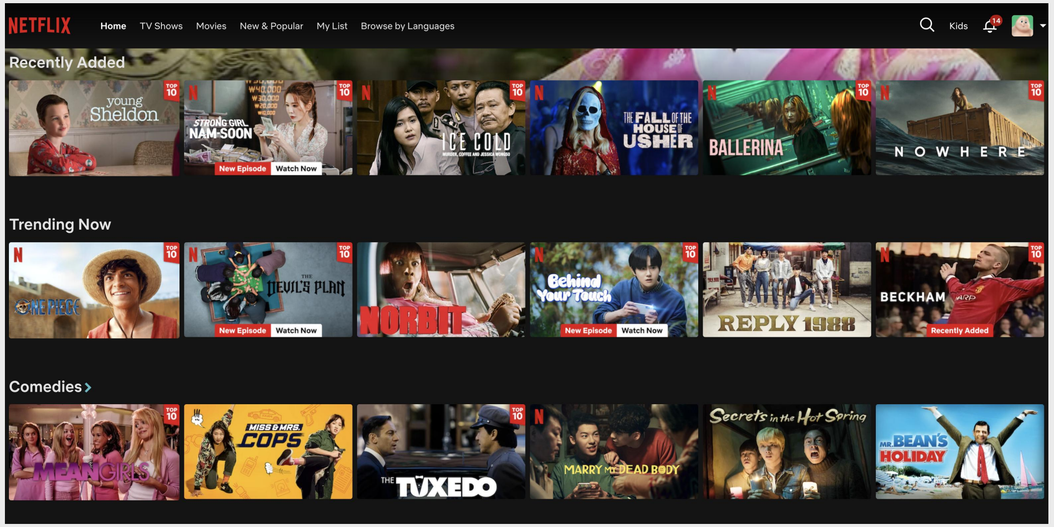
CASE STUDIES ON SOFTWARE ARCHITECTURE

NETFLIX - MICROSERVICES



CASE STUDIES ON SOFTWARE ARCHITECTURE

NETFLIX MICROSERVICES – APP BOOT



As of 2016

- Recommendations
- Trending Now
- Continue Watching
- My List
- Metrics

CASE STUDIES ON SOFTWARE ARCHITECTURE

MICROSOFT WORD - LAYERED ARCHITECTURE

- **Background**:
  - Utilizes a layered architecture to separate concerns.
- **Layers**:
  - UI Layer, Business Logic Layer, Data Access Layer.
- **Benefits**:
  - Improved maintainability and flexibility.

CASE STUDIES ON SOFTWARE ARCHITECTURE

DISCUSSION ON ARCHITECTURAL CHOICES AND OUTCOMES

- **Success Factors**:
  - Alignment with business goals.
  - Proper evaluation of trade-offs.
- **Lessons Learned**:
  - Importance of scalability considerations.
  - Need for thorough testing in distributed architectures.

ARCHITECTURAL DECISION-MAKING PROCESS

DISCUSSION ON ARCHITECTURAL CHOICES AND OUTCOMES

Stakeholder Involvement

- **Identifying Stakeholders**:
  - Developers, customers, end-users, project managers.
- **Gathering Requirements**:
  - Both functional and non-functional requirements.
- **Collaborative Decision-Making**:
  - Involving stakeholders in evaluating options.

Documentation and Communication

- **Architectural Descriptions**:
  - Use of UML diagrams, architectural blueprints.
- **Communication Tools**:
  - Documentation repositories, wikis, meetings.
- **Importance of Clarity**:
  - Ensures everyone has a shared understanding.

FIRST TIME WORKING WITH
ARCHITECTURE SOFTWARE
imgflip.com

REFERENCES

- Titus Winters, Tom Manshreck, and Hyrum Wright. (2020). ***Software Engineering at Google: Lesson Learned from Programming Over Time.*** 1st. O'Reilly Media. California. ISBN: 978-1492082798.
- Roger Pressman and Bruce Maxim. (2020). ***Software Engineering: A Practitioner's Approach.*** 9th. McGraw Hill. New York. ISBN: 978-1259872976.
- Ian Sommerville. (2016). ***Software Engineering.*** 10th. Pearson. London. ISBN: 978-1-292-09613-1.